



Rapport de Sprint

Développement Drupal - Custom Modules -



*Création de modules custom
Routes, Hooks, Plugins, Forms & Data Types*

Réalisé par

Hamza Lazaar

Encadré par

Abdelmajid Bendrif
Hamza Bahlaouane

Table des matières

1 Bases du Développement de Modules	4
1.1 Création des Premiers Modules	4
1.1.1 Module <code>hello_world</code>	4
1.1.2 Module <code>anytown</code>	5
1.2 Tests de Comportement du Module	6
2 Jour 2 : Routes, Contrôleurs, Services et Injection de Dépendances	7
2.1 Routes et Contrôleurs	7
2.1.1 Gestion des Menus	7
2.1.2 Récupération des Query Strings	7
2.2 Services et Conteneur de Services	8
2.2.1 Le Conteneur de Services Drupal	8
2.2.2 Injection de Dépendances	8
2.2.3 Guzzle et Logger	8
2.3 Templates Twig et Rendu	8
2.3.1 Retourner un Template Twig depuis un Contrôleur	8
2.3.2 Ajout de JavaScript Externe	9
2.4 Traduction et Internationalisation	9
2.4.1 Traduction côté PHP	9
2.4.2 Traduction côté JavaScript	9
2.5 Drush PHP et Services	9
2.6 Réponse JSON dans un Contrôleur	10
3 Jour 3 : Hooks	11
3.1 Principe des Hooks	11
3.2 Hooks Implémentés	11
3.2.1 <code>hook_form_alter</code> – Altération de Formulaires	11
3.2.2 Preprocess Hook – Variable <code>is_front</code>	12
3.2.3 <code>hook_page_attachments_alter</code> – Ajout de Metatag	13
3.2.4 <code>hook_preprocess_menu</code> – Classe CSS Personnalisée	14
3.2.5 <code>hook_preprocess_block</code> – Logo Personnalisé	14
4 Jour 4 : Plugins et Forms API	15
4.1 Système de Plugins	15
4.1.1 Création d'un Bloc Plugin	15
4.2 Forms API	16
4.2.1 Formulaire de Configuration (<code>ConfigFormBase</code>)	16
4.2.2 Validation de Formulaire	17
4.2.3 Rendu d'un Formulaire dans un Bloc	17
4.2.4 Redirection et Messages	17

4.2.5	Contrôle d'Accès avec <code>#access</code>	18
4.2.6	Groupement de Champs	18

5 Jour 5 : Data Types et Entity Queries 20

5.1	Configuration et Schéma	20
5.1.1	Dump de Configuration via Drush	20
5.1.2	Importance du Fichier <code>schema.yml</code>	20
5.2	Manipulation des Entités	20
5.2.1	Chargement et Manipulation de Nodes	20
5.2.2	Contraintes (Constraints)	21
5.2.3	View Builders et Modes d’Affichage	21
5.2.4	Entity Queries et <code>accessCheck</code>	21
5.2.5	Traduction d’un Node	22
5.3	Exercice Pratique : Module <code>node_reference</code>	22
5.3.1	Formulaire avec Entity Reference	22
5.3.2	Bloc avec Entity Queries	23
5.3.3	Hook Theme et Template Twig	24
5.3.4	Configuration et Schéma du Module	25

Conclusion 27

| Chapitre 1: Bases du Développement de Modules

1.1. Création des Premiers Modules

1.1.1. Module `hello_world`

Le premier module créé est `hello_world`, un module minimal servant de point d'entrée dans le développement Drupal. Sa structure est la suivante :

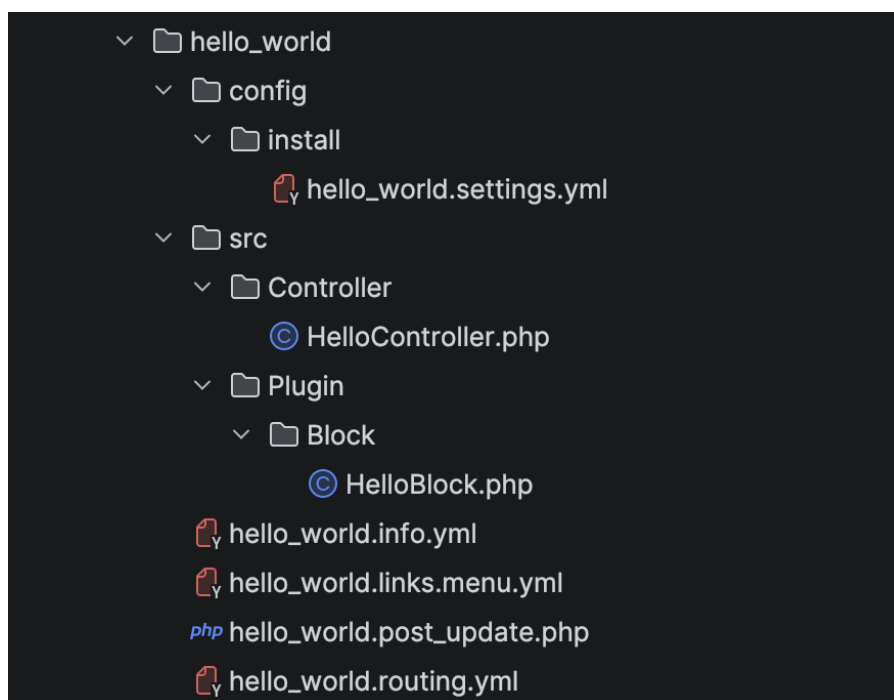


FIGURE 1.1 – Structure du Custom module Hello_world

Le fichier `hello_world.info.yml` déclare le module auprès de Drupal :

```
1 name: 'Hello World'
2 type: module
3 description: 'A simple Hello World module.'
4 package: Custom
5 core_version_requirement: ^10 || ^11
```

Listing 1.1 – `hello_world.info.yml`

Add [contributed modules](#) to extend your site's functionality.

Regularly review [available updates](#) and update as required to maintain a secure and current site. Always run the [update s](#)

The screenshot shows the Drupal module installation page. At the top, there is a 'Filter' input field containing the text 'Hello'. Below this, a section titled 'Custom' is expanded, showing a list of modules. The 'Hello World' module is checked with a checkbox. To its right, the text 'My first Drupal module.' is visible, followed by 'Machine name: hello_world'. At the bottom left of the module list, there is a blue 'Install' button.

FIGURE 1.2 – Module Hello World installé

1.1.2. Module anytown

Le second module, **anytown**, est plus complet et sert de base pour le reste du Sprint. Il implémente :

- Un **contrôleur** avec une page météo (`WeatherPage`)
- Un **formulaire de configuration** (`SettingsForm`)
- Un **bloc** personnalisé (`HelloWorldBlock`)
- Des **routes** et **liens de menu**
- Un **service** (`ForecastClient`) et son interface
- Des **permissions** personnalisées
- Des fichiers de **configuration** (`config/install`, `config/schema`)

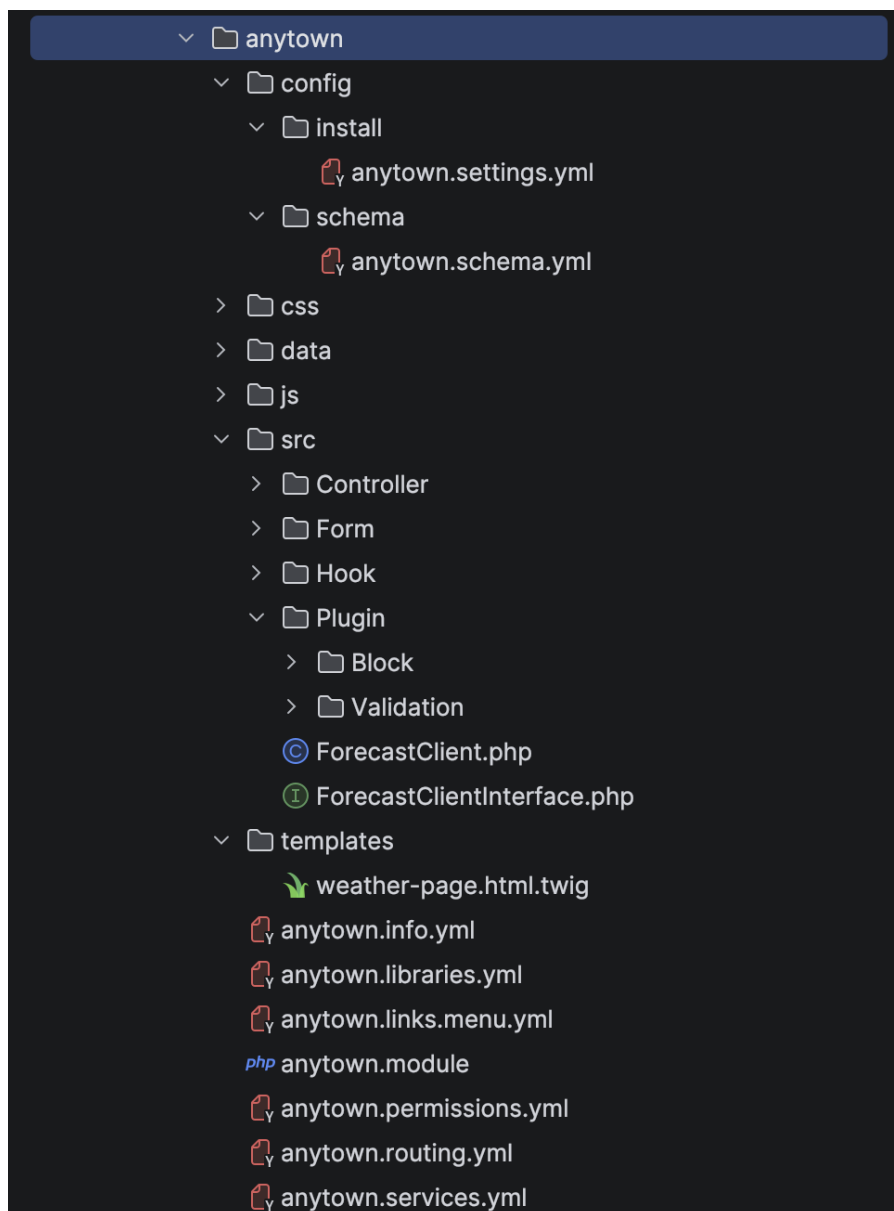


FIGURE 1.3 – Structure du module Anytown

1.2. Tests de Comportement du Module

Plusieurs tests ont été effectués pour comprendre le cycle de vie d'un module :

- **Désinstallation/Réinstallation** : La configuration du module est supprimée lors de la désinstallation et recrée à partir de `config/install/` lors de la réinstallation.
- **Modification de `core_version_requirement`** : Réduire la version à une version incompatible empêche l'activation du module.
- **Ajout de dépendances** : Ajouter `workflows` aux dépendances nécessite que ce module soit installé au préalable.
- **Suppression/Déplacement** : Drupal affiche un avertissement et les logs (`drush ws`) indiquent que le module est manquant.

| Chapitre 2: Jour 2 : Routes, Contrôleurs, Services et Injection de Dépendances

2.1. Routes et Contrôleurs

Le système de routage de Drupal repose sur le composant **Symfony Routing**. Chaque route est définie dans un fichier `.routing.yml` et connecte un chemin URL à un contrôleur.

```
1 anytown.weather_page:
2   path: '/weather/{style}'
3   defaults:
4     _title: 'Weather at the market'
5     _controller: '\Drupal\anytown\Controller\WeatherPage::build'
6     style: 'short'
7   requirements:
8     _permission: 'view weekly weather'
```

Listing 2.1 – Exemple de route (anytown.routing.yml)

2.1.1. Gestion des Menus

Le contrôle des menus se fait via le fichier `.links.menu.yml`. Le **weight** (poids) permet de trier l'ordre d'affichage des éléments. Les menus enfants sont créés en spécifiant un **parent** :

```
1 anytown.settings:
2   title: 'Anytown Settings'
3   route_name: anytown.settings
4   parent: system.admin_config_system
5   weight: 10
```

Listing 2.2 – Exemple de lien de menu avec poids et parent

2.1.2. Récupération des Query Strings

Dans un contrôleur, les paramètres de requête (query strings) sont récupérés via l'objet `Request` de Symfony :

```
1 use Symfony\Component\HttpFoundation\Request;
2
3 public function build(Request $request) {
4   $value = $request->query->get('param_name');
5 }
```

Listing 2.3 – Récupération d'un query string dans un contrôleur

2.2. Services et Conteneur de Services

2.2.1. Le Conteneur de Services Drupal

Les services définis par le cœur de Drupal se trouvent dans le fichier `core/core.services.yml`. Un module peut déclarer ses propres services via un fichier `.services.yml`.

```
1 services:
2   anytown.forecast_client:
3     class: Drupal\anytown\ForecastClient
4     arguments: ['@http_client', '@logger.factory']
```

Listing 2.4 – Déclaration d'un service (anytown.services.yml)

2.2.2. Injection de Dépendances

L'injection de dépendances dans un service se fait via le constructeur. Les arguments sont préfixés par `@` pour référencer d'autres services :

```
1 public function __construct(
2   ClientInterface $httpClient,
3   LoggerChannelFactoryInterface $loggerFactory
4 ) {
5   $this->httpClient = $httpClient;
6   $this->logger = $loggerFactory->get('anytown');
7 }
```

Listing 2.5 – Injection de dépendances dans un service

2.2.3. Guzzle et Logger

Guzzle est le client HTTP utilisé par Drupal pour effectuer des requêtes externes (service `http_client`). Le **Logger** permet d'enregistrer des messages dans les logs Drupal, consultables via `/admin/reports/dblog` ou `drush ws`. Le logger a également été utilisé dans le bloc `'catch'` pour afficher les messages d'erreurs.

2.3. Templates Twig et Rendu

2.3.1. Retourner un Template Twig depuis un Contrôleur

Un contrôleur retourne un render array utilisant un `#theme` enregistré via `hook.theme()` :

```
1 return [
2   '#theme' => 'weather_page',
3   '#weather_intro' => $intro,
4   '#weather_forecast' => $forecast,
5 ];
```

Listing 2.6 – Retour d'un template Twig dans un contrôleur

2.3.2. Ajout de JavaScript Externe

L'attachement de bibliothèques externes se fait via les **libraries** Drupal et le mécanisme `#attached` :

```
1 # anytown.libraries.yml
2 tailwind:
3   js:
4     https://unpkg.com/@tailwindcss/browser@4: { type: external }
```

Listing 2.7 – Ajout de JS externe via libraries

2.4. Traduction et Internationalisation

2.4.1. Traduction côté PHP

La fonction `$this->t()` rend une chaîne traduisible. Pour rechercher la traduction dans l'interface d'administration (`/admin/config/regional/translate`), on utilise le texte source (ex : `Hello, @name!`) :

```
1 $this->t('Hello, @name!', ['@name' => $username]);
2 // Rechercher "Hello" dans l'interface de traduction
```

Listing 2.8 – Chaîne traduisible en PHP

2.4.2. Traduction côté JavaScript

En JavaScript, la fonction `Drupal.t()` est utilisée pour rendre les chaînes traduisibles :

```
1 Drupal.t('Hello, @name!', {'@name': username});
```

Listing 2.9 – Chaîne traduisible en JavaScript

2.5. Drush PHP et Services

L'outil `drush php` permet d'interagir avec le conteneur de services :

```
1 # R cup rer le service path.alias_manager
2 drush php:eval "\$alias = \Drupal::service('path_alias.manager')
3   ->getAliasByPath('/node/1'); print_r(\$alias);"
4
5 # G n rer l'URL d'une route
6 drush php:eval "use Drupal\Core\Url;
7   print_r(Url::fromRoute('anytown.weather_page')
8   ->setAbsolute()->toString());"
```

Listing 2.10 – Utilisation de drush php pour les services

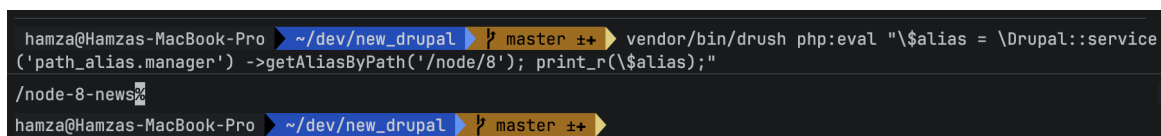


FIGURE 2.1 – Test du service `path.alias_manager` avec drush

2.6. Réponse JSON dans un Contrôleur

Pour retourner une réponse JSON au lieu d'un rendu HTML :

```
1 use Symfony\Component\HttpFoundation\JsonResponse;
2
3 public function apiEndpoint() {
4     return new JsonResponse([
5         'status' => 'ok',
6         'data' => $data,
7     ]);
8 }
```

Listing 2.11 – Réponse JSON dans un contrôleur

Chapitre 3: Jour 3 : Hooks

3.1. Principe des Hooks

Les hooks Drupal sont un mécanisme permettant aux modules d'intervenir dans le fonctionnement de Drupal à des points précis du cycle d'exécution. Drupal 10 supporte deux syntaxes : les fonctions procédurales dans le fichier `.module` et les classes avec l'attribut PHP 8 `#[Hook]`.

3.2. Hooks Implémentés

3.2.1. `hook_form_alter` – Altération de Formulaires

Ce hook permet de modifier n'importe quel formulaire Drupal. Il reçoit trois arguments : `$form` (le render array du formulaire), `$form_state` (l'état du formulaire) et `$form_id` (l'identifiant).

```
1 function anytown_form_alter(  
2     array &$amp;form,  
3     FormStateInterface $form_state,  
4     $form_id  
5 ) {  
6     if ($form_id === 'some_target_form') {  
7         $form['my_custom_field'] = [  
8             '#type' => 'textfield',  
9             '#title' => t('My Custom Field'),  
10            '#description' => t('Enter a value.'),  
11            '#weight' => 50,  
12        ];  
13    }  
14 }
```

Listing 3.1 – Implémentation de `hook_form_alter` (anytown.module)

```
no usages  hamza-lzr

#[Hook('form_user_register_form_alter')]
public function formUserRegisterFormAlter(&$form, FormStateInterface $form_state) : void {
    // Add our custom validation handler.
    $form['#validate'][] = 'anytown_user_register_validate';

    $form['terms_of_use'] = [
        '#type' => 'fieldset',
        '#title' => t('Anytown Terms and Conditions of Use'),
        '#weight' => 10,
        // Admin users can skip the terms of use, this will let them create accounts
        // for other people without seeing these fields.
        '#access' => !\Drupal::currentUser()->hasPermission('administer users')
    ];

    $form['terms_of_use']['terms_of_use_data'] = [
        '#type' => 'markup',
        '#markup' => '<p>By checking the box below you agree to our terms of use. Whatever that might be. ~_{ツ}_/~</p>',
    ];

    $form['terms_of_use']['terms_of_use_checkbox'] = [
        '#type' => 'checkbox',
        '#title' => t('I agree with the terms above'),
        '#required' => TRUE,
    ];
}
```

FIGURE 3.1 – Hook qui implémente form alter, pour modifier le formulaire de création de compte

FIGURE 3.2 – Formulaire de user register, modifié avec un Hook

3.2.2. Preprocess Hook – Variable is_front

Le hook preprocess permet d'ajouter des variables aux templates Twig. En utilisant le service path.matcher, la variable is_front est passée à tous les templates :

```
1 function my_module_preprocess(array &$variables) {
2     $variables['is_front'] = \Drupal::service('path.matcher')
```

```

3   ->isFrontPage();
4   }

```

Listing 3.2 – Ajout de la variable is_front

La variable peut ensuite être utilisée et vérifiée dans un template Twig avec `{{ dump(is_front) }}`.

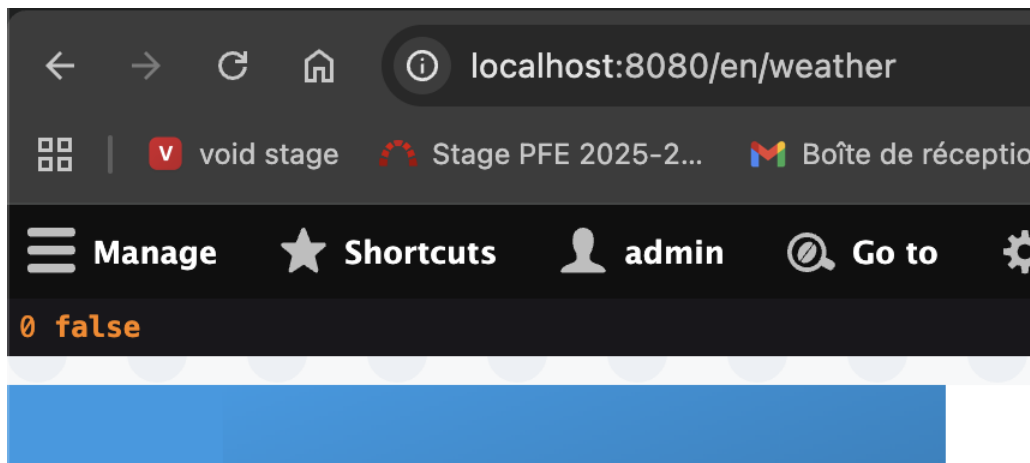


FIGURE 3.3 – Dump de la variable isFront dans la page /weather

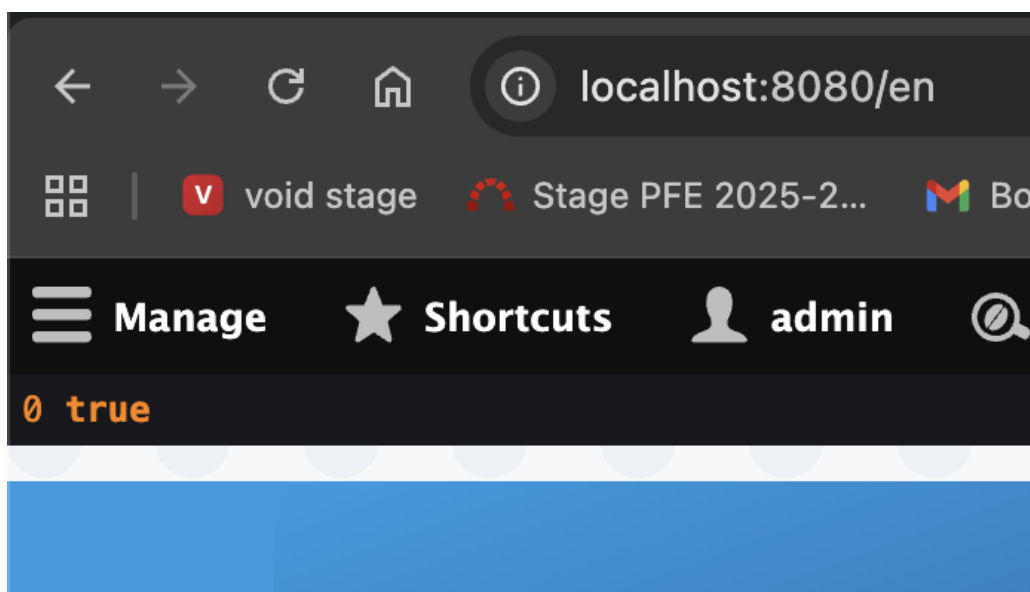


FIGURE 3.4 – Dump de la variable isFront dans la page principale /

3.2.3. hook_page_attachments_alter – Ajout de Metatag

Ce hook permet d'ajouter des éléments au `<head>` de la page. Ici, un metatag viewport est injecté :

```

1 function anytown_page_attachments_alter(array &$attachments) {
2     $viewport = [
3         '#tag' => 'meta',
4         '#attributes' => [
5             'name' => 'viewport',
6             'content' => 'width=device-width, initial-scale=1,

```

```
7         shrink-to-fit=no',
8     ],
9 ];
10 $attachments['#attached']['html_head'][] = [
11     $viewport, 'viewport'
12 ];
13 }
```

Listing 3.3 – Ajout du metatag viewport

3.2.4. hook_preprocess_menu – Classe CSS Personnalisée

Ce hook ajoute une classe CSS `my-custom-class` à tous les éléments de menu :

```
1 function anytown_preprocess_menu(array &$variables) {
2     foreach ($variables['items'] as &$item) {
3         $item['attributes']->addClass('my-custom-class');
4     }
5 }
```

Listing 3.4 – Ajout de classe CSS aux menus

3.2.5. hook_preprocess_block – Logo Personnalisé

Ce hook modifie le bloc `system_branding_block` pour remplacer le logo du site :

```
1 function anytown_preprocess_block(array &$variables) {
2     if ($variables['plugin_id'] === 'system_branding_block') {
3         $variables['elements']['content']['site_logo']['#uri'] =
4             'https://static.cdnlogo.com/logos/d/88/drupal-wordmark.svg';
5     }
6 }
```

Listing 3.5 – Remplacement du logo du site

| Chapitre 4: Jour 4 : Plugins et Forms API

4.1. Système de Plugins

Les plugins Drupal constituent un design pattern permettant l'extensibilité. Un plugin est une classe PHP qui implémente une interface spécifique et est découverte automatiquement par Drupal via des **annotations** ou des **attributs PHP 8**.

4.1.1. Création d'un Bloc Plugin

Le module **anytown** inclut un bloc **HelloWorldBlock** illustrant le pattern plugin avec injection de dépendances :

```
1  #[Block(
2      id: 'anytown_hello_world',
3      admin_label: new TranslatableMarkup(
4          'Hello World, from the anytown block'
5      ),
6      category: new TranslatableMarkup('Custom')
7  )]
8  class HelloWorldBlock extends BlockBase
9      implements ContainerFactoryPluginInterface {
10
11      private $current_user;
12
13      public function __construct(
14          array $configuration,
15          $plugin_id,
16          $plugin_definition,
17          AccountProxyInterface $current_user,
18      ) {
19          parent::__construct($configuration, $plugin_id,
20              $plugin_definition);
21          $this->current_user = $current_user;
22      }
23
24      public static function create(
25          ContainerInterface $container,
26          array $configuration,
27          $plugin_id,
28          $plugin_definition
29      ) {
30          return new static(
```

```

31     $configuration, $plugin_id, $plugin_definition,
32     $container->get('current_user')
33 );
34 }
35
36 public function build(): array {
37     $name = $this->current_user->getDisplayName();
38     if ($this->current_user->isAuthenticated()) {
39         $build['content'] = [
40             '#markup' => $this->t('Hello, %name !',
41                 ['%name' => $name]),
42         ];
43     } else {
44         $build['content'] = [
45             '#markup' => $this->t('Hello, World')
46         ];
47     }
48     return $build;
49 }
50 }

```

Listing 4.1 – Bloc plugin avec attribut PHP 8

Hello World, from the anytime block

Hello, *admin* !

FIGURE 4.1 – Affichage du bloc, Hello, admin! car connecté avec le username admin

4.2. Forms API

4.2.1. Formulaire de Configuration (ConfigFormBase)

Le module anytown inclut un formulaire de configuration (SettingsForm) qui étend ConfigFormBase :

```

1 final class SettingsForm extends ConfigFormBase {
2     const SETTINGS = 'anytown.settings';
3
4     public function buildForm(array $form,
5         FormStateInterface $form_state): array {
6
7         $form['display_forecast'] = [

```



```

8      '#type' => 'checkbox',
9      '#title' => $this->t('Display weather forecast'),
10     '#default_value' => $this->config(self::SETTINGS)
11         ->get('display_forecast'),
12 ];
13 $form['location'] = [
14     '#type' => 'textfield',
15     '#title' => $this->t('ZIP code of market'),
16     '#default_value' => $this->config(self::SETTINGS)
17         ->get('location'),
18     '#placeholder' => '90210',
19 ];
20 return parent::buildForm($form, $form_state);
21 }
22 }

```

Listing 4.2 – Formulaire de configuration anytown

4.2.2. Validation de Formulaire

La validation se fait dans la méthode `validateForm()`. Les hooks (`hook_form_alter`) permettent également d'ajouter une validation externe :

```

1 public function validateForm(array &$form,
2     FormStateInterface $form_state): void {
3     $location = $form_state->getValue('location');
4     $value = filter_var($location, FILTER_VALIDATE_INT);
5     if (!$value || strlen((string) $value) !== 5) {
6         $form_state->setErrorByName('location',
7             $this->t('Invalid ZIP code'));
8     }
9 }

```

Listing 4.3 – Validation du formulaire

4.2.3. Rendu d'un Formulaire dans un Bloc

Pour afficher un formulaire dans la méthode `build()` d'un bloc, on utilise le service `form_builder` :

```

1 public function build(): array {
2     return \Drupal::formBuilder()
3         ->getForm('Drupal\mymodule\Form\MyForm');
4 }

```

Listing 4.4 – Rendu d'un formulaire dans un bloc

4.2.4. Redirection et Messages

Après la soumission d'un formulaire, la redirection est gérée via `setRedirect()` et les messages via `messenger()` :

```

1 public function submitForm(array &$form,
2     FormStateInterface $form_state) {
3     // Message de succes (vert)
4     $this->messenger()->addMessage(

```

```
5     $this->t('Configuration saved.')
6   );
7   // Redirection vers une route
8   $form_state->setRedirect('anytown.weather_page');
9 }
```

Listing 4.5 – Redirection et message de succes

4.2.5. Contrôle d'Accès avec #access

La propriété #access permet de masquer un champ pour les utilisateurs anonymes :

```
1 $form['admin_field'] = [
2   '#type' => 'textfield',
3   '#title' => $this->t('Admin only field'),
4   '#access' => \Drupal::currentUser()->isAuthenticated(),
5 ];
```

Listing 4.6 – Masquer un champ pour les anonymes

4.2.6. Groupement de Champs

Les champs peuvent être regroupés avec fieldset ou details :

```
1 $form['group'] = [
2   '#type' => 'details',
3   '#title' => $this->t('Advanced settings'),
4   '#open' => FALSE,
5 ];
6 $form['group']['field_1'] = [
7   '#type' => 'textfield',
8   '#title' => $this->t('Field 1'),
9 ];
```

Listing 4.7 – Groupement de champs

Home > Administration > Configuration > System

Anytown Settings ☆

✓ **Status message**

Anytown configuration updated.

☒ Display weather forecast

ZIP code of market

77665

Used to determine weekend weather forecast.

Weather-related closures

closure 5

List one closure per line.

Save configuration

FIGURE 4.2 – Validation du formulaire anytown settings

| Chapitre 5: Jour 5 : Data Types et Entity Queries

5.1. Configuration et Schéma

5.1.1. Dump de Configuration via Drush

La commande `drush php` permet d'inspecter la configuration de n'importe quel module :

```
1 drush php:eval "print_r(\Drupal::config(  
2   'eu_cookie_compliance.settings')->getRawData());"
```

Listing 5.1 – Dump de la configuration eu_cookie_compliance

Les fichiers de configuration installés par les modules se trouvent dans le répertoire `config/install/` de chaque module. Le fichier de configuration actif est stocké dans la table `config` de la base de données.

5.1.2. Importance du Fichier `schema.yml`

Le fichier `schema.yml` est essentiel pour :

- **Validation** : Vérifier la conformité des valeurs de configuration
- **Traduction** : Identifier les chaînes traduisibles dans la config
- **Documentation** : Décrire la structure attendue de la configuration
- **Import/Export** : Valider les données lors de la synchronisation de configuration

```
1 anytown.settings:  
2   type: config_object  
3   label: 'Anytown settings'  
4   mapping:  
5     display_forecast:  
6       type: boolean  
7       label: 'Toggle forecast display on/off'  
8     location:  
9       type: string  
10      label: 'ZIP code where the market takes place'
```

Listing 5.2 – Exemple de schéma de configuration

5.2. Manipulation des Entités

5.2.1. Chargement et Manipulation de Nodes

```
1 use Drupal\node\Entity\Node;
2
3 // Charger un node
4 $node = Node::load(1);
5
6 // Lire les valeurs des champs
7 $title = $node->getTitle();
8 $body = $node->get('body')->value;
9 $field_value = $node->get('field_name')->value;
10
11 // Modifier un champ et sauvegarder
12 $node->set('title', 'Nouveau titre');
13 $node->set('field_name', 'Nouvelle valeur');
14 $node->save();
```

Listing 5.3 – Chargement et manipulation d’un node

5.2.2. Contraintes (Constraints)

Les **Constraints** sont un mécanisme de validation au niveau des entités. Elles permettent de définir des règles de validation sur les champs d’une entité, par exemple vérifier qu’un nom d’utilisateur est unique ou qu’une valeur est dans un intervalle donné.

Le module **anytown** inclut un exemple de contrainte personnalisée (**UserNameConstraint**) avec son validateur.

5.2.3. View Builders et Modes d’Affichage

Les **View Builders** sont responsables du rendu des entités dans différents modes d’affichage (**full**, **teaser**, etc.). Le mode **teaser** est un affichage condensé d’un contenu, typiquement utilisé dans les listes de contenu.

```
1 $view_builder = \Drupal::entityTypeManager()
2   ->getViewBuilder('node');
3 $build = $view_builder->view($node, 'teaser');
```

Listing 5.4 – Rendu d’un node en mode teaser

5.2.4. Entity Queries et accessCheck

Le rôle de **accessCheck()** dans les entity queries est de vérifier les permissions d’accès de l’utilisateur actuel avant de retourner les résultats. Mis à **TRUE**, seuls les contenus auxquels l’utilisateur a accès sont retournés.

```
1 $nids = \Drupal::entityTypeManager()
2   ->getStorage('node')
3   ->getQuery()
4   ->condition('type', 'article')
5   ->condition('status', 1)
6   ->accessCheck(TRUE)
7   ->execute();
```

Listing 5.5 – Entity query avec accessCheck

5.2.5. Traduction d'un Node

Pour obtenir la traduction d'un node dans une langue spécifique :

```
1 $node = Node::load(1);
2 if ($node->hasTranslation('fr')) {
3     $node_fr = $node->getTranslation('fr');
4     $title_fr = $node_fr->getTitle();
5 }
```

Listing 5.6 – Obtenir la traduction française d'un node

5.3. Exercice Pratique : Module node_reference

L'exercice final du Day 5 constitue la synthèse de tous les apprentissages. Un nouveau module `node_reference` a été créé, combinant : formulaire avec entity reference, entity queries, bloc plugin, et hook_theme avec template Twig.

5.3.1. Formulaire avec Entity Reference

Un formulaire de configuration (`NodeReferenceSettingsForm`) a été créé avec un champ `entity_autocomplete` permettant de sélectionner un node :

```
1 final class NodeReferenceSettingsForm extends ConfigFormBase {
2     const SETTINGS = 'node_reference.settings';
3
4     public function buildForm(array $form,
5         FormStateInterface $form_state): array {
6         $config = $this->config(self::SETTINGS);
7         $nid = $config->get('selected_node');
8         $default_node = $nid ? Node::load($nid) : NULL;
9
10        $form['selected_node'] = [
11            '#type' => 'entity_autocomplete',
12            '#title' => $this->t('Select a node'),
13            '#target_type' => 'node',
14            '#default_value' => $default_node,
15            '#required' => TRUE,
16        ];
17        return parent::buildForm($form, $form_state);
18    }
19
20    public function submitForm(array &$form,
21        FormStateInterface $form_state): void {
22        $this->config(self::SETTINGS)
23            ->set('selected_node',
24                (int) $form_state->getValue('selected_node'))
25            ->save();
26        $this->messenger()->addMessage(
27            $this->t('Node reference configuration saved.'));
28        parent::submitForm($form, $form_state);
29    }
30 }
```

Listing 5.7 – Formulaire avec entity_autocomplete (NodeReferenceSettingsForm.php)

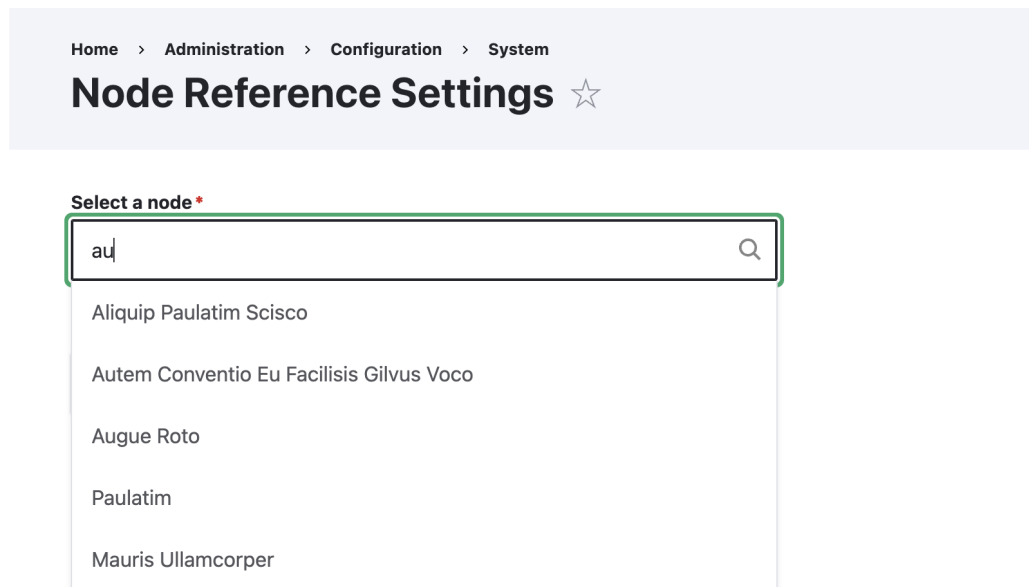


FIGURE 5.1 – Selection de nodes avec autocomplete

5.3.2. Bloc avec Entity Queries

Le bloc `NodeReferenceBlock` charge le node sélectionné, récupère son type de contenu, puis effectue une entity query pour trouver les nodes du même type (en excluant le node sélectionné) :

```

1 public function build(): array {
2     $config = $this->configFactory
3     ->get('node_reference.settings');
4     $nid = $config->get('selected_node');
5     $selected_node = Node::load($nid);
6     $selected_title = $selected_node->getTitle();
7     $bundle = $selected_node->bundle();
8
9     // Entity query : meme type, exclure le nid actuel
10    $node_storage = $this->entityTypeManager
11    ->getStorage('node');
12    $query = $node_storage->getQuery()
13    ->condition('type', $bundle)
14    ->condition('nid', $nid, '<>')
15    ->condition('status', 1)
16    ->accessCheck(TRUE)
17    ->range(0, 10)
18    ->sort('created', 'DESC');
19
20    $result_nids = $query->execute();
21    $related_titles = [];
22    if (!empty($result_nids)) {
23        $related_nodes = $node_storage
24        ->loadMultiple($result_nids);
25        foreach ($related_nodes as $node) {
26            $related_titles[] = $node->getTitle();
27        }

```

```

28 }
29
30 return [
31   '#theme' => 'node_reference_block',
32   '#selected_title' => $selected_title,
33   '#related_nodes' => $related_titles,
34   '#cache' => [
35     'tags' => ['node:' . $nid, 'node_list',
36               'config:node_reference.settings'],
37   ],
38 ];
39 }

```

Listing 5.8 – Entity query dans le bloc (NodeReferenceBlock.php)

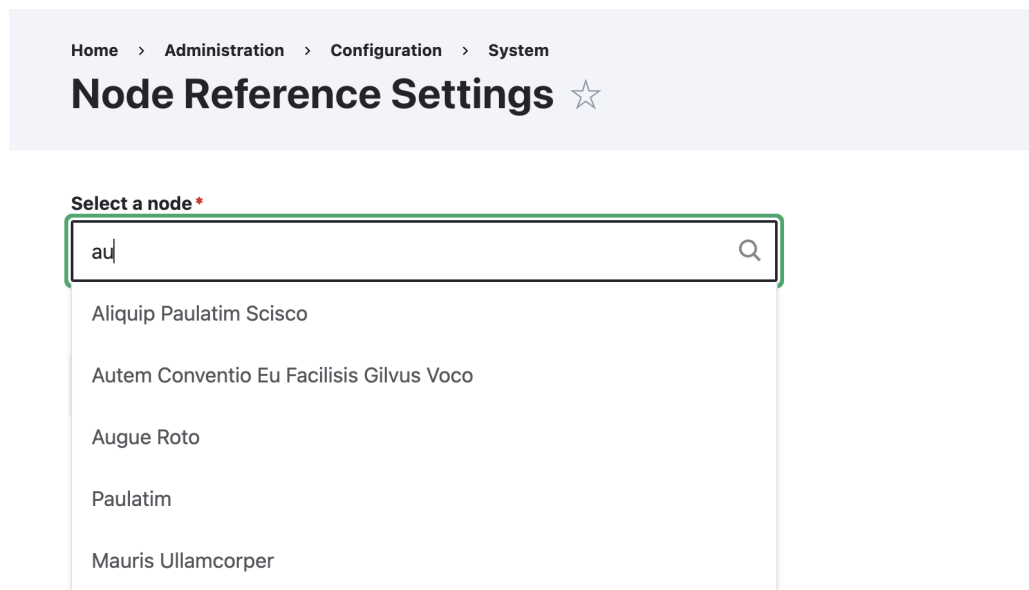


FIGURE 5.2 – Affichage du bloc, qui montre la node sélectionnée et des nodes avec le même Content Type

5.3.3. Hook Theme et Template Twig

Le hook `hook_theme()` est implémenté via l'attribut PHP 8 `#[Hook('theme')]` pour enregistrer le template :

```

1 class NodeReferenceTheme {
2   #[Hook('theme')]
3   public function theme(): array {
4     return [
5       'node_reference_block' => [
6         'variables' => [
7           'selected_title' => '',
8           'related_nodes' => [],
9         ],
10      ],
11    ];

```



```

12 }
13 }

```

Listing 5.9 – Implémentation de hook_theme (NodeReferenceTheme.php)

Le template Twig correspondant affiche le titre du node sélectionné et la liste des nodes du même type :

```

1 <div class="node-reference-block">
2   <h3>{{ "Selected Node"|t }}</h3>
3   <p>{{ selected_title }}</p>
4
5   {% if related_nodes is not empty %}
6     <h4>{{ "Related Nodes (same content type)"|t }}</h4>
7     <ul>
8       {% for title in related_nodes %}
9         <li>{{ title }}</li>
10      {% endfor %}
11    </ul>
12  {% else %}
13    <p>{{ "No other nodes found."|t }}</p>
14  {% endif %}
15 </div>

```

Listing 5.10 – Template Twig (node-reference-block.html.twig)

5.3.4. Configuration et Schéma du Module

```

1 node_reference.settings:
2   type: config_object
3   label: 'Node Reference settings'
4   mapping:
5     selected_node:
6       type: integer
7       label: 'The NID of the selected node'
8       nullable: true

```

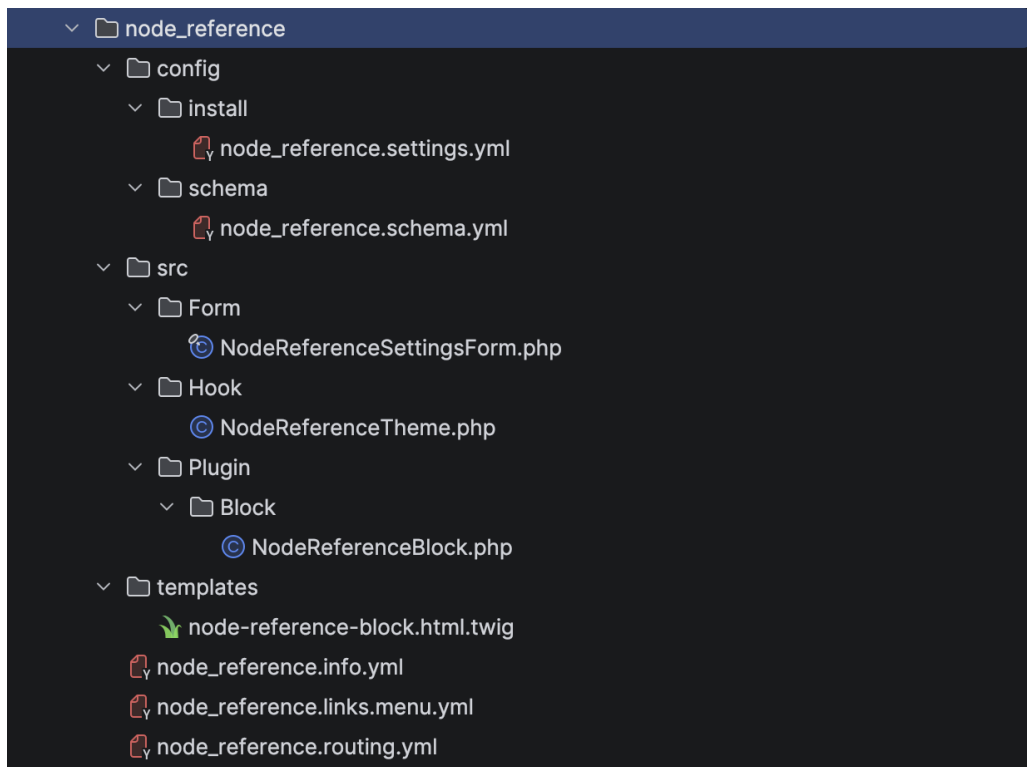
Listing 5.11 – Schéma de configuration (node_reference.schema.yml)

```

1 node_reference.settings:
2   path: '/admin/config/system/node-reference'
3   defaults:
4     _title: 'Node Reference Settings'
5     _form: 'Drupal\node_reference\Form\
6           NodeReferenceSettingsForm'
7   requirements:
8     _permission: 'administer site configuration'

```

Listing 5.12 – Route du formulaire (node_reference.routing.yml)

FIGURE 5.3 – Structure du module `node_reference`

| Chapitre 5: Conclusion

Ce sprint de développement Drupal a permis de couvrir les fondamentaux du développement de modules custom, en progressant systématiquement des bases vers des concepts avancés :

- **Jour 1** : Structure modulaire, fichiers de déclaration (`.info.yml`), cycle de vie des modules
- **Jour 2** : Routes, contrôleurs, services, injection de dépendances, rendu Twig et traduction
- **Jour 3** : Hooks (`form_alter`, `preprocess`, `page_attachments_alter`), intervention dans le pipeline de rendu
- **Jour 4** : Plugins (blocs), Forms API complète (validation, redirection, access control, groupement)
- **Jour 5** : Data types, entity queries, configuration avec schéma, et exercice de synthèse

Trois modules custom ont été développés (`hello_world`, `anytown`, `node_reference`), chacun illustrant des concepts progressivement plus avancés. Le module final `node_reference` constitue une synthèse pratique maîtrisant l'ensemble des API Drupal étudiées.